

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5
6         if (sc.hasNextInt()) {
7             int inputNumber = sc.nextInt();
8
9             TaskThread t1 = new TaskThread("Prime", inputNumber);
10            TaskThread t2 = new TaskThread("Armstrong", inputNumber);
11            TaskThread t3 = new TaskThread("Reverse", inputNumber);
12
13            t1.start();
14            try { t1.join(); } catch (InterruptedException e) { }
15
16            t2.start();
17            try { t2.join(); } catch (InterruptedException e) { }
18
19            t3.start();
20            try { t3.join(); } catch (InterruptedException e) { }
21        }
22        sc.close();
23    }
24 }
25
26 class TaskThread extends Thread {
27     private int number;
28
29     public TaskThread(String name, int number) {
```

153

Output

```
Prime Thread: 153 is not a prime
number
Armstrong Thread: 153 is an
Armstrong number
Reverse Thread: Reverse of 153 =
351
```

Visible Test Cases

- | | |
|-------------|---|
| Test Case 1 | ✓ |
| Test Case 2 | ✓ |
| Test Case 3 | ✓ |

10.00 / 10 marks

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         if (sc.hasNextInt()) {
8             int num = sc.nextInt();
9
10            PrimeThread t1 = new PrimeThread(num);
11            FactorialThread t2 = new FactorialThread(num);
12            ReverseThread t3 = new ReverseThread(num);
13
14            try {
15
16                t1.start();
17                t1.join();
18
19
20                t2.start();
21                t2.join();
22
23
24                t3.start();
25                t3.join();
26            } catch (InterruptedException e) {
27                System.out.println(e.getMessage());
28            }
29        }
30    }
```

7

Output

```
Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7
```

Visible Test Cases

- | | |
|-------------|---|
| Test Case 1 | ✓ |
| Test Case 2 | ✓ |
| Test Case 3 | ✓ |

20.00 / 20 marks

```
1 public class Main {
2     public static void main(String[] args) throws InterruptedException {
3         Counter sharedCounter = new Counter();
4
5         ThreadA t1 = new ThreadA(sharedCounter);
6         ThreadB t2 = new ThreadB(sharedCounter);
7
8         t1.start();
9         t2.start();
10
11
12         t1.join();
13         t2.join();
14
15         System.out.println("Final Count = " + sharedCounter.getCount());
16     }
17 }
18
19 class Counter {
20     private int count = 0;
21
22
23     public synchronized void increment() {
24         count++;
25     }
26
27     public int getCount() {
28         return count;
29     }
30 }
```

stdin input

Output

```
ThreadA completed
ThreadB completed
Final Count = 2000
```

Visible Test Cases

- | | |
|-------------|---|
| Test Case 1 | ✓ |
| Test Case 2 | ✓ |
| Test Case 3 | ✓ |

10.00 / 10 marks